

Vendor-neutral SDD Framework

Evidence-based System Design

STATUS
Proposed

OWNER
Framework Architecture

LAST UPDATED
August 02, 2026

Szerzők	OpenAI Codex — source-backed multi-agent research
Reviewerek	Framework owner; security; developer experience; platform adapters
Kapcsolódó anyagok	Öt jelölti dosszié (S1/S2 source/runtime evidence) és a reusable-pattern katalógus (S3 source-to-design mapping)
Scope	Javasolt S3 Global–Project–Session–Local taxonomy, agentek, workflowk, adapterek, state, security és operations

1. Összefoglaló

Az öt vizsgált SDD framework bizonyítékai alapján egy kis, vendorsemleges orchestration mag javasolt. A mag typed artifact graphot, role- és workflow-szerződéseket, eventeket, tartós futási naplót és ellenőrzött materialization transactiont biztosít; a Codex, Claude Code és más környezetek csak adapterként jelennek meg.

A rendszer ember-agent együttműködésre, sub-agent delegálásra, több sessionön átívelő folytatásra és auditálható projektartefaktumokra optimalizált. Nem helyettesít modellt vagy IDE-t, nem szintetizál jóváhagyást, és nem tekinti a promptot biztonsági sandboxnak. Minden side effect deklarált capabilityhez, rögzített bemenethez és megfigyelhető terminal state-hez kötött. Evidence status: a jelölti viselkedések S1/S2 bizonyítékok; a közös scope-ok, contractok, könyvtárnevek és SLO-k S3 javasolt design.

2. Célok és nem-célok

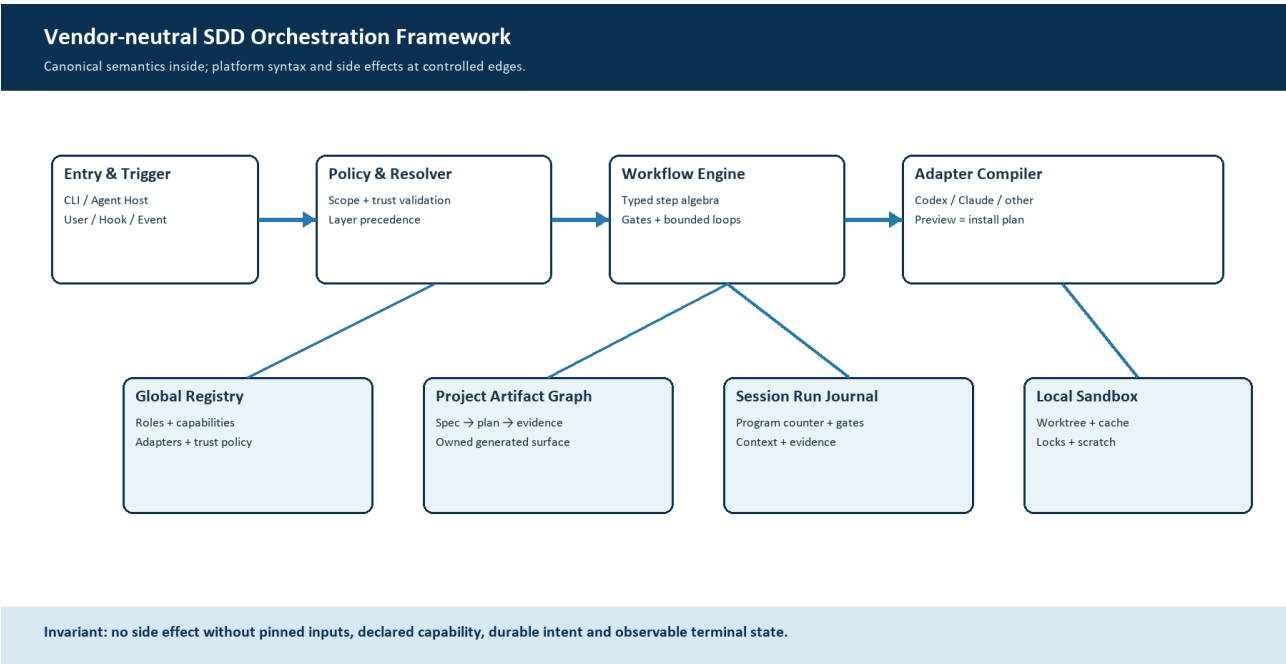
Célok	Nem-célok
Deterministic artifact graph és resumable workflow állapot	Saját foundation model vagy IDE készítése
Vendorsemleges canonical IR és tesztelt adapterek	A promptot biztonsági sandboxként kezelni
Auditálható role, capability, event és approval contract	Automatikus jóváhagyás, push, merge vagy production módosítás
Safe preview/install/update/recovery/uninstall lifecycle	Harmadik fél teljes frameworkjének változatlan átvétele

3. Háttér és probléma

A jelöltek külön-külön erős részmegoldásokat adnak: OpenSpec artifact DAGot és root provenance-t; Spec Kit adaptereket, ownershipöt és security utilityket; GSD descriptor registryt, producer/checker loopokat és consentet; BMAD reprodukálható execution packageket és rétegzett konfigurációt; Paul könnyen tanulható PLAN–APPLY–UNIFY ciklust. Egyik teljes átvétele sem gazdaságos: a nagy rendszerek erősen csatoltak; Paulnál 58 concrete installed reference-nek nincs distributed targetje, miközben a tényleges Claude runtime failure nem lett reprodukálva.

A rendszerhatár belsejében canonical IR, schema, policy, artifact graph és run journal található. A platform-specifikus fájlformátum, hook, parancs és tool invocation az adapterperem felelőssége. Alapelv: canonical semantics inside, platform syntax at the edges; generált fájl csak hash- és ownership-bizonyítékkal írható felül.

4. Javasolt architektúra



1. ábra. Vendorsemleges SDD orchestration architektúra és scope-rétegek.

Fő komponensek

Komponens	Felelősség	Elsődleges tárolás	Hibaviselkedés
Canonical Registry & IR	Schema, role, workflow, event, capability és package verziók	Global registry + project lock	Ismeretlen verzió vagy reference: fail closed
Policy & Resolver	Layer precedence, trust, scope, consent és effective config	Policy store + project manifest	Bizonytalan scope/permission: execution tiltás
Workflow & Event Engine	Typed step algebra, program counter, gate, bounded loop, fan-out/fan-in	Session run journal	Atomikus checkpoint; explicit paused/failed state
Adapter Compiler	Preview/materialize/update/remove platform-native fájlokkal	Staging + ownership manifest	Unowned overwrite/symlink/drift: abort és rollback
Artifact, Evidence & Observability	Project DAG, evidence, metrics, logs, traces és recovery	Project state + append-only audit	State-write hiba előtt nincs downstream side effect

5. Kérés- és workflow-életciklus

- A User, CLI, native hook vagy agent event létrehoz egy versioned RunEnvelope-ot: event type, scope, actor, correlation ID és kívánt workflow.
- A boundary guard validálja a schemát, realpath-scope-ot, trust source-ot, engedélyezett capabilityket és az adapter támogatását; bizonytalan scope esetén fail closed.
- A javasolt S3 resolver kulcsenként rögzíti a Global/Project/Session/Local forrást és precedence-et; a Global trust/policy kulcsok nem felülírhatók. Ezután pineli az artifact graph verzióit, a role contractot, a package lockot és az ownership manifestet. A logical owner és a fizikai fájlhely eltérhet.
- A Workflow Engine typed step algebra alapján Sequence, Gate, Branch, bounded Loop, FanOut/FanIn, AgentStep, ToolStep és ArtifactWrite elemeket ütemez.
- Side effect előtt atomikusan rögzítendő a program counter, pinned input hashok, idempotency key, approval state, timeout/retry budget és a tervezett state transition.
- Az adapter csak deklarált toolt hívhat. Retry kizárólag besorolt, idempotens hibára engedett; minden loopnak max iteration, deadline és done/blocked/paused/failed terminal state-je van.
- A rendszer validálja az output artefaktumot, frissíti az artifact graphot, lezárja a run journal bejegyzést, és eventet, metricet, logot, trace-t, valamint ember számára olvasható evidence summaryt bocsát ki.

6. API- és adatszerződések

Elsődleges RunEnvelope szerződés

Mező	Típus	Kötelező	Leírás
schema_version	SemVer	Igen	RunEnvelope és transition contract verzió; ismeretlen major elutasítandó.
run_id / correlation_id	UUID	Igen	Egy futás és a kapcsolódó eventlánc stabil azonosítói.
scope	Enum	Igen	Global, Project, Session vagy Local; realpath-bound ownershipdel.
event	Object	Igen	Type, source, actor, timestamp, preconditions és requested workflow.
pinned_inputs	Hash map	Igen	Artifact, config, role, workflow, adapter és package verziók/hashok.
capabilities	Array	Igen	Engedélyezett tools, write scope, network, secrets, timeout és retry class.
transition	Object	Igen	From/to state, idempotency key, attempt, evidence és terminal reason.

Szerződéses garanciák

- Execution előtt a RunEnvelope, effective config, pinned artifact set és capability decision tartós és schema-valid.
- Minden write/run/attempt monotonic sequence-et, run ID-t, correlation ID-t és idempotency key-t kap.
- Audit/replay rögzíti a framework-, schema-, adapter-, role-, workflow-, config- és input-hash verziókat.
- A project artifact graph a delivery state source of truth; a run journal végrehajtási bizonyíték, nem üzleti specifikáció.

A versioned schema és adapter contract a projekt `framework/schemas/` és `framework/adapters/` könyvtárában publikált; minden contract release együtt verziózza őket.

7. Konzisztencia, idempotencia és replay

Project-state mutation single-writer lockkal és atomic replace-szel történik. Preview és install ugyanazt az immutable MaterializationPlan objektumot fogyasztja. Replay csak rögzített input- és config-hashokkal engedett; eltérés új run. Párhuzamos agentek külön output ownershipöt kapnak, majd explicit FanIn validálja az összefésülést.

Scenario	Várt viselkedés	Indoklás
Duplikált event vagy replay	Az idempotency key ugyanazt a terminal resultot adja; nincs új side effect.	A program counter és output ownership tartós.
Kötelező state write vagy dependency hibázik	Fail closed; retryability besorolt; downstream végrehajtás nem indul.	A durable intent megelőzi a side effectet.
Timeout vagy részleges adapterhiba	Bounded retry után failed/paused; journal recovery packetet tartalmaz.	A részállapot látható és rekonstruálható.
Config változik futás közben	A futás a pinned effective configot használja; új verzió új run.	Nincs mid-run semantic drift.

8. Biztonság és adatvédelem

- A capability policy allow-list alapú; scope, realpath, symlink és worktree ownership minden side effect előtt ellenőrzött.
- A context manifest csak szükséges artefaktumot tölt; secret, credential, PII és teljes tool output nem kerül promptba vagy logba.
- Credentialt a framework nem tárol projektfájlban; platform secret store-ból rövid élettartamú capabilityként kapja.
- Network, shell, destructive VCS, migration és external package alapból tiltott; preview, explicit approval, bounded scope és read-back szükséges.
- Run evidence retention policyvezérelt; append-only audit különül a törölhető scratch/cache-tól; deletion célpontja explicit és recovery-tervhez kötött.

9. Üzemeltetési készenlét

Signal	SLO vagy alert	Owner	Launch gate
Workflow terminal completion	>=99% kontrollált tesztcorpuson; 0 néma success	Core runtime	Kötelező
Checkpoint/materialization latency	P95 cél és workflow-class timeout dokumentált	Runtime + adapters	Ajánlott
Retry/fallback/error rate	Retry class és terminal reason szerint; fallback nem lehet néma	Ops	Kötelező
Generated/config drift	0 ownership-, hash-, registry- és adapter-parity eltérés	Platform adapters	Kötelező
Recovery/replay integrity	Crash-injection és restore teszt minden release-ben	Core + security	Kötelező

10. Vizsgált alternatívák

Alternatíva	Miért volt érdekes	Miért nem választottuk
OpenSpec közvetlen fork	A legtisztább artifact DAG és root provenance.	Önmagában nem ad elég erős generic workflow engine/materialization ownershipöt.
GitHub Spec Kit közvetlen fork	Érett adapter-, event-, security- és ownership réteg.	Nagy, termékspecifikus extension/preset monolitok és raw shell boundary.
GSD vagy BMAD teljes port	A leggazdagabb agent-, role-, loop-, registry- és customization rendszer.	Nagyon nagy kompatibilitási felület, prompt coupling és magas üzemeltetési költség.
Paul prompt-only megközelítés	A PLAN–APPLY–UNIFY ciklus könnyen tanulható.	58 concrete installed reference-nek nincs distributed targetje; a Claude runtime failure inference. Hiányzó CARL és nincs erős enforcement.

11. Nyitott döntések

- Első adapterpár: Codex + Claude Code, vagy egyetlen reference adapter a core stabilizálásáig?
- A human approval ledger személyhez, csapathoz vagy repository policyhoz legyen kötve?
- A state maradjon reviewable YAML/Markdown, vagy az append-only journal használjon SQLite-ot exportált nézettel?

d. Community package trusthez hash + explicit consent elegendő, vagy szervezeti aláírási lánc szükséges?

12. Döntés és következő lépések

Döntés: clean-room vendorsemleges mag építése. Direct reuse esetén az MIT notice megőrzése jogi kötelezettség; a repository/commit/file provenance megtartása külön belső auditpolicy. Az első mérföldkő egy egy-runtime-os, read-only artifact graph + dry-run materializer. Író pilot csak zero-diff parity, negative-path security, crash recovery és emberi approval bizonyíték után indulhat.

Mérföldkő	Deliverable	Kilépési feltétel
M1	Canonical schema, artifact graph, run journal, read-only CLI	Schema, DAG readiness és replay contract tesztek zöldék
M2	Egy reference adapter + pure MaterializationPlan dry-run	Preview/install parity, ownership, path/symlink és rollback tesztek zöldék
M3	Role/sub-agent engine, gates, bounded loops, evidence	Disposable end-to-end pilot; 0 néma fallback; recovery packet rekonstruálható
M4	Codex + Claude adapter, package catalog és operations	Canary SLO, uninstall/recovery rehearsal és security approval teljes